An Energy-Efficient Hardware Accelerator for Convolutional Neural Network on Zynq SoC

Project Report submitted to

APJ ABDUL KALAM TECHNOLOGICAL UNIIVERSITY

BACHELOR OF TECHNOLOGY
IN
ELECTTRONICS AND COMMUNICATION ENGINEERING

Submittedby :

ADITH M	**(MUT23EC002)**
ANANDHAKRISHNAN M	**(MUT23EC011)**
BENISON P BABY	**(MUT23EC019)**
SAYOOJ P V	**(MUT23EC067)**

DEPARTMENT OF ELECTRONICS & COMMUNICATION ENGINEERING
MUTHOOT INSTITUTE OF TECHNOLOGY AND SCIENCE
VARIKOLO P.O,PUTHENCRUZ-682308

MARCH 2026

MARCH 2026

MUTHOOT INSTITUTE OF TECHNOLOGY AND SCIENCE
VARIKOLI P.O, PUTHENCRUZ – 682308
DEPARTMENT OF ELECTRONICS & COMMUNICATION ENGINEERING

CERTIFICATE

This is to certify that the report entitled ***"An Energy-Efficient Hardware Accelerator for Convolutional Neural Network on Zynq SoC"*** *is a bonafide record of miniproject work done by* ***ADITH M(MUT23EC002), ANANDHAKRISHNAN M(MUT23EC011), BENISON P BABY (MUT23EC019), SAYOOJ P V (MUT23EC067)*** *during the year 2025–2026. This report is submitted to APJ Abdul Kalam Technological University in partial fulfilment of the requirements for the award of the degree of Bachelor of Technology in Electronics & Communication Engineering.*

Project Guides

Ms. Deepa Prabhu
Professor, ECE

Ms. Nayana P K
Asst. Professor, ECE

Project Coordinators

Ms. Jisha Jacob
Asst. Professor, ECE

Ms. Divya R S
Asst. Professor, ECE

Dr. Abhilash Antony
Professor
HOD, ECE

Contents

1 INTRODUCTION **5**
- 1.1 General Introduction . . . 5
- 1.2 Motivation and Scope . . . 5
- 1.3 Objectives . . . 6

2 LITERATURE REVIEW **7**
- 2.1 Deep Learning for Image Classification . . . 7
- 2.2 Limitations of CPU and GPU-Based Implementations . . . 7
- 2.3 FPGA-Based CNN Accelerators . . . 7
- 2.4 Quantization Techniques in Hardware Acceleration . . . 8
- 2.5 Energy Optimization Techniques . . . 8
- 2.6 Research Gap . . . 8

3 METHODOLOGY **10**
- 3.1 Overview of Proposed System . . . 10
- 3.2 Dataset and Preprocessing . . . 10
- 3.3 CNN Architecture . . . 11
- 3.4 Convolution Operation . . . 11
- 3.5 Activation Function (ReLU) . . . 12
- 3.6 Pooling Operation (Max-Pooling) . . . 12
- 3.7 Fully Connected Layer . . . 12
- 3.8 Post-Training Quantization . . . 13
- 3.9 Hardware Mapping Strategy . . . 13
- 3.10 Energy Optimization using Clock Gating . . . 13

4 HARDWARE ARCHITECTURE **15**
- 4.1 Overview of System Architecture . . . 15
- 4.2 Hardware-Software Co-Design . . . 15
- 4.3 AXI Interface Communication . . . 16
- 4.4 Processing Element (PE) Architecture . . . 16
- 4.5 Memory Optimization using Shift Buffers . . . 16
- 4.6 Finite State Machine (FSM) Controller . . . 17
- 4.7 Pipelined Datapath . . . 17
- 4.8 Power Optimization Techniques . . . 17

5 RESULTS AND DISCUSSION **19**
- 5.1 Overview of Results . . . 19
- 5.2 Functional Verification . . . 19
- 5.3 Hardware Output Analysis . . . 20

5.4 Power Consumption Analysis 21
5.5 Memory and Resource Utilization 21
5.6 Performance Evaluation 22
5.7 Comparison with Existing Systems 22
5.8 Discussion 22

6 CONCLUSION AND FUTURE SCOPE **24**
6.1 Conclusion 24
6.2 Future Scope 24

ABSTRACT

The rapid growth of Artificial Intelligence (AI) applications in edge and embedded systems has created a strong demand for energy-efficient hardware solutions. Convolutional Neural Networks (CNNs), although highly effective for image classification tasks, require significant computational resources and memory bandwidth, making them unsuitable for direct deployment on power-constrained devices.

This project presents the design and implementation of an energy-efficient hardware accelerator for CNN inference on the Xilinx Zynq-7000 System-on-Chip (SoC). The proposed architecture targets real-time handwritten digit recognition using the MNIST dataset while minimizing power consumption and hardware resource utilization.

To overcome memory and computational bottlenecks, a custom hardware architecture is developed using parallel Processing Elements (PEs), shift-buffer-based data reuse techniques, and on-chip memory optimization. The design leverages a hybrid hardware-software co-design approach, where preprocessing is handled in software and computationally intensive CNN operations are executed in FPGA fabric.

Furthermore, a Sparsity-Aware Dynamic Clock Gating (SADCG) technique is incorporated to exploit the inherent sparsity introduced by ReLU activation, effectively reducing unnecessary switching activity and dynamic power consumption. Post-training quantization is applied to convert floating-point parameters into INT8 format, ensuring efficient mapping onto FPGA resources.

The proposed system is implemented and verified using Xilinx Vivado tools and MATLAB-based validation. Experimental results demonstrate successful real-time classification with improved energy efficiency, reduced memory access overhead, and optimized hardware utilization.

This work highlights the effectiveness of FPGA-based CNN accelerators for edge AI applications and provides a scalable framework for low-power, real-time intelligent systems.

LIST OF SYMBOLS

Symbol	Description
W	Input feature map dimension
K	Kernel (filter) size
P	Padding applied to input
S	Stride of convolution
O	Output feature map dimension
$X[m, n]$	Input feature map value at position (m, n)
$W[i, j]$	Convolution kernel weight
$Y[m, n]$	Output feature map value
B	Bias value
W_{FP}	Floating-point weights
W_{INT8}	Quantized 8-bit weights
S_W	Scaling factor for quantization
$f(x)$	Activation function (ReLU)

Chapter 1

INTRODUCTION

1.1 General Introduction

Artificial Intelligence (AI) has emerged as a transformative technology in modern electronic systems, enabling machines to perform complex tasks such as image recognition, speech processing, and decision-making. Among various AI models, Convolutional Neural Networks (CNNs) have achieved state-of-the-art performance in image classification and pattern recognition tasks due to their ability to automatically extract spatial features from input data.

However, the deployment of CNN models on edge devices such as embedded systems, IoT nodes, and wearable electronics is highly challenging. These models require intensive computation involving repeated convolution operations and large-scale memory access. Traditional implementations using CPUs and GPUs consume significant power and introduce latency, making them unsuitable for real-time, low-power applications.

To address these challenges, hardware accelerators have been developed to offload computationally intensive tasks from general-purpose processors. Field Programmable Gate Arrays (FPGAs) provide a promising platform for such accelerators due to their reconfigurability, parallel processing capability, and energy efficiency. Unlike software-based implementations, FPGA-based designs enable custom datapaths that match the algorithmic flow, thereby improving throughput and reducing power consumption.

In this project, an energy-efficient hardware accelerator for Convolutional Neural Network inference is designed and implemented on the Xilinx Zynq-7000 System-on-Chip (SoC). The system adopts a hybrid hardware-software co-design approach, where preprocessing tasks are handled by the ARM processor, and computationally intensive CNN operations are executed in FPGA fabric. The proposed design targets real-time handwritten digit classification using the MNIST dataset, ensuring both accuracy and efficiency.

1.2 Motivation and Scope

The rapid growth of edge computing has increased the demand for real-time AI processing directly on local devices rather than relying on cloud-based systems. Although cloud computing provides high computational capability, it suffers from several limitations such as latency, privacy concerns, and dependency on continuous internet connectivity.

CNN models, while highly accurate, are computationally expensive due to large num-

bers of Multiply-Accumulate (MAC) operations and high memory bandwidth requirements. This leads to increased power consumption and thermal constraints, especially in embedded environments.

The motivation behind this project is to develop a hardware-efficient solution that overcomes these limitations by:

- Reducing power consumption through optimized hardware design
- Minimizing memory access using data reuse techniques
- Enabling real-time processing on edge devices
- Improving computational efficiency using parallel processing

The scope of this project includes the design and implementation of a CNN accelerator using FPGA technology, specifically targeting low-power and real-time applications. The proposed system can be extended to various domains such as smart surveillance systems, healthcare monitoring devices, and IoT-based intelligent systems.

1.3 Objectives

The primary objective of this project is to design and implement an energy-efficient CNN hardware accelerator on the Zynq SoC platform. The specific objectives are as follows:

- To design a CNN model capable of handwritten digit classification
- To implement convolution, activation, and pooling operations in hardware
- To optimize memory usage using shift buffers and on-chip storage
- To reduce power consumption using clock gating techniques
- To apply post-training quantization for efficient hardware mapping
- To achieve real-time inference with minimal latency

Chapter 2

LITERATURE REVIEW

2.1 Deep Learning for Image Classification

Deep Learning has significantly advanced the field of image classification, with Convolutional Neural Networks (CNNs) emerging as the most effective models. CNNs are capable of automatically extracting spatial and hierarchical features from input images through convolutional, pooling, and fully connected layers.

Several state-of-the-art CNN architectures have demonstrated high accuracy in image classification tasks. However, these models typically rely on dense floating-point computations and require substantial computational resources. This results in high power consumption and increased latency, making them unsuitable for deployment in edge devices and embedded systems.

2.2 Limitations of CPU and GPU-Based Implementations

Traditional implementations of CNNs rely on CPUs and GPUs. CPUs offer flexibility but lack the parallel processing capability required for high-speed CNN inference. GPUs, on the other hand, provide massive parallelism and are widely used for training deep neural networks.

Despite their advantages, GPUs consume significant power, often exceeding the limits of battery-powered devices. Additionally, GPU-based systems require complex memory hierarchies and cooling mechanisms, making them impractical for compact embedded applications.

Furthermore, both CPU and GPU implementations often depend on cloud-based processing, which introduces latency, increases communication overhead, and raises privacy concerns in sensitive applications.

2.3 FPGA-Based CNN Accelerators

Field Programmable Gate Arrays (FPGAs) have emerged as a promising alternative for implementing CNN accelerators in edge environments. Unlike CPUs and GPUs, FPGAs

allow the creation of custom hardware architectures tailored to specific algorithms.

Several FPGA-based CNN accelerators have been proposed to improve performance and energy efficiency. These designs exploit parallelism by implementing multiple Processing Elements (PEs) and utilize on-chip memory to reduce external memory access.

However, a major challenge in FPGA-based implementations is the memory bandwidth limitation. Standard Block RAM (BRAM) architectures provide limited read/write ports, which can lead to data starvation in convolution operations. To overcome this, techniques such as shift buffers and sliding window mechanisms have been introduced to reuse data efficiently.

2.4 Quantization Techniques in Hardware Acceleration

High-precision floating-point operations are inefficient for hardware implementation due to their high resource usage. Quantization techniques are widely used to convert floating-point weights and activations into lower precision formats such as INT8.

Post-Training Quantization (PTQ) is a commonly used approach that maps trained model parameters into fixed-point representations without retraining. This significantly reduces memory usage and enables efficient implementation on FPGA hardware.

However, aggressive quantization may lead to accuracy degradation if not carefully designed. Therefore, a balance between precision and efficiency must be maintained.

2.5 Energy Optimization Techniques

Power consumption is a critical factor in edge AI systems. One of the most effective techniques for reducing dynamic power is clock gating, which disables the clock signal to inactive components.

In CNNs, activation functions such as ReLU introduce sparsity by converting negative values to zero. This sparsity can be exploited using Sparsity-Aware Dynamic Clock Gating (SADCG), where computations involving zero values are skipped, reducing unnecessary switching activity.

Several research works have demonstrated that such techniques can significantly reduce power consumption without affecting system performance.

2.6 Research Gap

Although significant progress has been made in FPGA-based CNN accelerators, several challenges remain:

- High power consumption due to inefficient utilization of computational resources
- Memory bottlenecks caused by limited BRAM bandwidth
- Lack of effective exploitation of sparsity in hardware

- Trade-off between accuracy and efficiency in quantized models

To address these challenges, this project proposes an optimized CNN hardware accelerator that integrates data reuse techniques, parallel processing, quantization, and sparsity-aware clock gating to achieve high performance with low power consumption.

Chapter 3

METHODOLOGY

3.1 Overview of Proposed System

The proposed system is an energy-efficient hardware accelerator designed to perform Convolutional Neural Network (CNN) inference on an FPGA platform. The system follows a hybrid hardware-software co-design approach, where preprocessing operations are executed in software and computationally intensive CNN operations are implemented in hardware.

The complete workflow of the system consists of the following stages:

- Input image acquisition and preprocessing
- Normalization of input data
- CNN inference (convolution, activation, pooling, and classification)
- Output prediction of handwritten digits (0–9)

The system is implemented on the Xilinx Zynq-7000 SoC, where the ARM Cortex-A9 processor handles control operations, and the programmable logic (FPGA fabric) performs parallel CNN computations.

3.2 Dataset and Preprocessing

The system is validated using the MNIST dataset, which is a standard benchmark dataset for handwritten digit recognition. The dataset consists of grayscale images of size 28×28 pixels representing digits from 0 to 9.

Before feeding the data into the CNN model, preprocessing steps are performed to improve the efficiency and accuracy of the system:

- **Grayscale Conversion:** Input images are converted into grayscale format if required.
- **Resizing:** Images are resized to 28×28 pixels to match the CNN input size.
- **Normalization:** Pixel values are scaled from the range [0, 255] to [0, 1] using Min-Max normalization.

Normalization improves convergence during training and ensures numerical stability during inference.

3.3 CNN Architecture

The Convolutional Neural Network used in this project consists of multiple layers designed to extract features and perform classification. The main components of the CNN are:

- Convolutional Layer
- Activation Function (ReLU)
- Pooling Layer (Max-Pooling)
- Fully Connected Layer

Each layer performs a specific function in transforming the input image into a final classification output.

3.4 Convolution Operation

The convolution operation is the core computation in CNNs. It involves sliding a kernel (filter) over the input feature map to extract local features such as edges, textures, and patterns.

The output feature map dimension is calculated using:

$$O = \frac{W - K + 2P}{S} + 1$$

where:

- W = Input dimension
- K = Kernel size
- P = Padding
- S = Stride

The convolution operation is mathematically expressed as:

$$Y[m, n] = \sum_{i=0}^{K-1} \sum_{j=0}^{K-1} X[m + i, n + j] \cdot W[i, j] + B$$

where:

- X = Input feature map
- W = Kernel weights
- Y = Output feature map
- B = Bias

This operation is implemented in hardware using parallel Processing Elements (PEs) to improve computational efficiency.

3.5 Activation Function (ReLU)

After convolution, a non-linear activation function is applied to introduce non-linearity into the network. In this project, the Rectified Linear Unit (ReLU) is used due to its simplicity and hardware efficiency.

The ReLU function is defined as:

$$f(x) = \max(0, x)$$

This means that all negative values are converted to zero, while positive values remain unchanged.

From a hardware perspective, ReLU is highly efficient because it can be implemented using simple comparator logic instead of complex mathematical operations. Additionally, ReLU introduces sparsity in the output feature maps by producing a large number of zero values, which can be exploited for reducing unnecessary computations.

3.6 Pooling Operation (Max-Pooling)

Pooling is used to reduce the spatial dimensions of feature maps, thereby decreasing computational complexity and memory usage.

In this project, Max-Pooling is used, which selects the maximum value from a defined window (typically 2×2) of the input feature map.

The operation is defined as:

$$Y_{pool} = \max(X[i, j])$$

Max-Pooling provides translation invariance and reduces the size of intermediate data, making the system more efficient for hardware implementation.

3.7 Fully Connected Layer

The fully connected (FC) layer performs the final classification by mapping the extracted features into output classes.

Each neuron in the fully connected layer is connected to all neurons in the previous layer. The operation involves a weighted sum followed by bias addition:

$$Y = \sum X_i W_i + B$$

The output of this layer represents the probability scores for each digit class (0–9), and the class with the highest score is selected as the final prediction.

3.8 Post-Training Quantization

Floating-point computations are inefficient for FPGA implementation due to high resource usage. Therefore, Post-Training Quantization (PTQ) is applied to convert floating-point weights into fixed-point INT8 representation.

The quantization process is given by:

$$W_{INT8} = clip\left(round\left(\frac{W_{FP}}{S_W}\right), -128, 127\right)$$

where:

- W_{FP} = Floating-point weights
- S_W = Scaling factor
- W_{INT8} = Quantized weights

This reduces memory usage and enables efficient mapping onto FPGA hardware without significantly affecting accuracy.

3.9 Hardware Mapping Strategy

The CNN model is mapped onto FPGA hardware using a custom architecture designed for parallel processing and efficient memory usage.

Key hardware strategies include:

- **Parallel Processing Elements (PEs):** Multiple convolution operations are executed simultaneously to increase throughput.
- **Shift Buffer Mechanism:** Data reuse is achieved by storing input pixels in shift registers, reducing redundant memory access.
- **On-Chip Memory Utilization:** Intermediate data is stored in Block RAM (BRAM) to minimize external memory access.
- **Pipelined Architecture:** Different stages of computation are overlapped to improve overall processing speed.

3.10 Energy Optimization using Clock Gating

To reduce dynamic power consumption, Sparsity-Aware Dynamic Clock Gating (SADCG) is implemented.

Since ReLU produces many zero values, computations involving zero inputs can be skipped. A zero-detection logic identifies such cases and disables the clock signal to inactive hardware blocks.

This reduces switching activity in digital circuits, leading to significant power savings without affecting system performance.

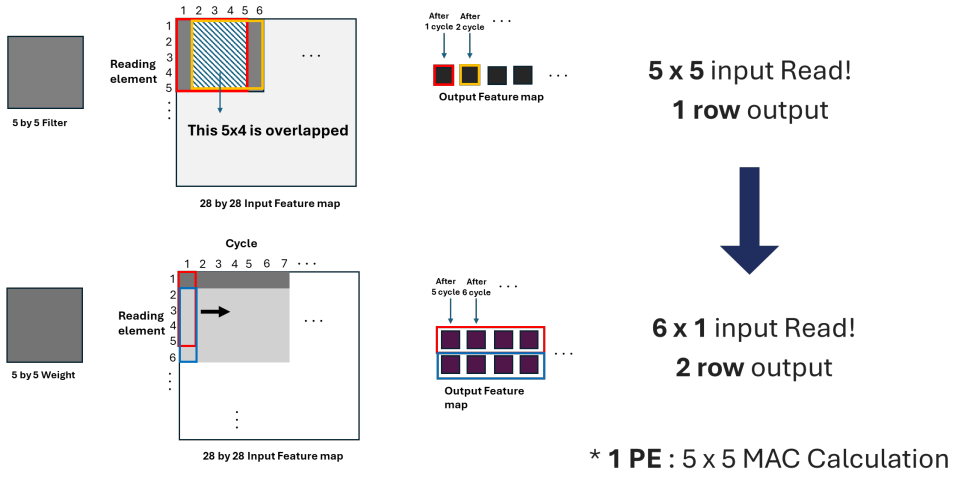


Figure 3.1: Memory access minimization using shift buffers and overlapping sliding window technique in PE array.

Chapter 4

HARDWARE ARCHITECTURE

4.1 Overview of System Architecture

The proposed system is implemented on the Xilinx Zynq-7000 System-on-Chip (SoC), which integrates a Processing System (PS) and Programmable Logic (PL) on a single chip. This enables an efficient hardware-software co-design approach for executing Convolutional Neural Network (CNN) inference.

The Processing System (PS), consisting of an ARM Cortex-A9 processor, is responsible for control operations, preprocessing, and data management. The Programmable Logic (PL) implements the CNN accelerator, performing computationally intensive operations such as convolution, activation, and pooling.

The interaction between PS and PL is achieved through the Advanced eXtensible Interface (AXI), ensuring efficient data transfer and synchronization.

4.2 Hardware-Software Co-Design

The system is partitioned into software and hardware domains to optimize performance and efficiency.

- **Software (PS):**
  - Input data preprocessing
  - Normalization of image data
  - Control signal generation
  - Data transfer to FPGA
- **Hardware (PL):**
  - Convolution operations
  - Activation (ReLU)
  - Pooling operations
  - Fully connected computations

This partitioning reduces computational load on the processor and accelerates inference speed.

4.3 AXI Interface Communication

The communication between the Processing System and Programmable Logic is implemented using the AXI interface.

Instead of using complex Direct Memory Access (DMA), a memory-mapped AXI interface is used due to the relatively small size of input data. This reduces overhead and simplifies system design.

The data transfer process includes:

- Writing input data from PS to PL
- Triggering computation using control signals
- Monitoring status signals (start, busy, done)
- Reading output results from PL

This ensures synchronized and efficient communication between hardware and software components.

4.4 Processing Element (PE) Architecture

The core computation of the CNN accelerator is performed using Processing Elements (PEs). Each PE is responsible for executing Multiply-Accumulate (MAC) operations required for convolution.

Multiple PEs operate in parallel, significantly increasing throughput. Each PE includes:

- Multiplier unit
- Adder unit
- Register storage

Parallel execution of PEs enables faster computation compared to sequential processing.

4.5 Memory Optimization using Shift Buffers

Memory access is a critical bottleneck in hardware accelerators. To overcome this limitation, shift buffer mechanisms are used.

Instead of repeatedly fetching data from memory, input pixels are stored in shift registers and reused across multiple convolution operations. This significantly reduces memory bandwidth requirements.

Shift buffers act as local caches, enabling efficient data reuse and minimizing redundant memory access.

4.6 Finite State Machine (FSM) Controller

The overall operation of the hardware accelerator is controlled by a Finite State Machine (FSM). The FSM ensures proper sequencing of operations and synchronization between different modules.

The main states of the FSM include:

- IDLE: Waiting for start signal
- LOAD: Loading input data and weights
- COMPUTE: Performing CNN operations
- WRITEBACK: Storing output results

The FSM guarantees deterministic execution and efficient control flow within the system.

4.7 Pipelined Datapath

To improve performance, the hardware architecture employs pipelining, where multiple stages of computation are executed simultaneously.

The pipeline stages include:

- Data loading
- Convolution computation
- Activation and pooling
- Output generation

Pipelining increases throughput and reduces latency by overlapping operations.

4.8 Power Optimization Techniques

To ensure energy efficiency, several optimization techniques are incorporated:

- **Clock Gating:** Disables clock signals to inactive modules, reducing dynamic power consumption.
- **Sparsity Exploitation:** Zero values generated by ReLU are used to skip unnecessary computations.

- **Reduced Memory Access:** Shift buffers minimize power consumption associated with memory operations.

These techniques collectively improve the energy efficiency of the system while maintaining performance.

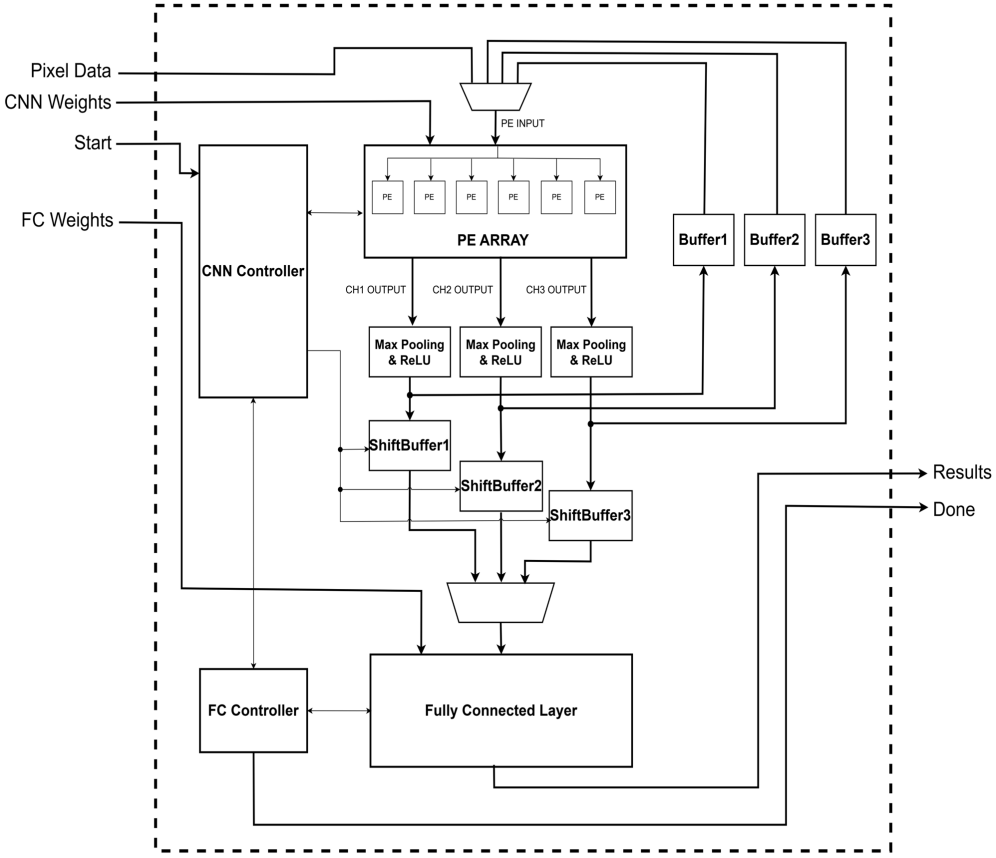


Figure 4.1: Overall System Architecture of the CNN Hardware Accelerator showing PS-PL interaction via AXI interface.

Chapter 5

RESULTS AND DISCUSSION

5.1 Overview of Results

The proposed CNN hardware accelerator was successfully implemented on the Xilinx Zynq-7000 SoC and verified using simulation and hardware testing. The system demonstrates efficient real-time classification of handwritten digits from the MNIST dataset.

The results validate the correctness of convolution, activation, pooling, and classification operations implemented in the FPGA.

5.2 Functional Verification

Functional verification was carried out using simulation tools and hardware validation.

- The CNN layers (Convolution, ReLU, and Pooling) were tested individually to ensure correct operation.
- Intermediate signals were observed to verify correct data flow between layers.
- Final output classification was validated against expected results.

Simulation waveforms confirmed that the hardware implementation matches the expected behavior of the CNN model.

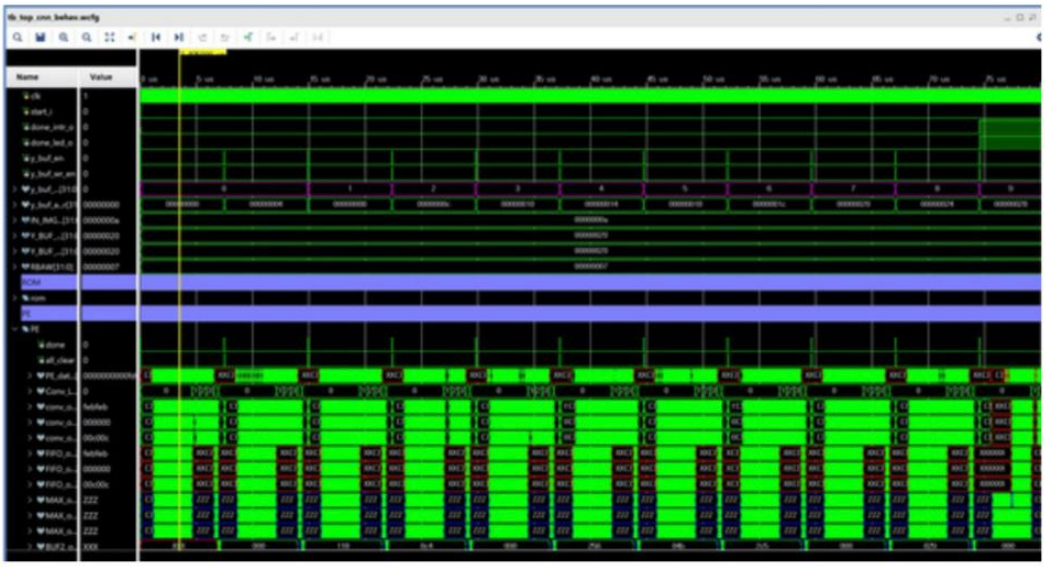


Figure 5.1: Simulation waveform showing FSM states, control signals, and parallel MAC operations during CNN inference.

5.3 Hardware Output Analysis

The hardware accelerator successfully classified handwritten digits (0–9) in real time. The output was obtained through the FPGA after processing the input image.

The classification results demonstrate:

- Correct prediction of input digits
- Stable and consistent output across multiple test cases
- Real-time inference capability

The system effectively performs end-to-end CNN inference within the FPGA environment.

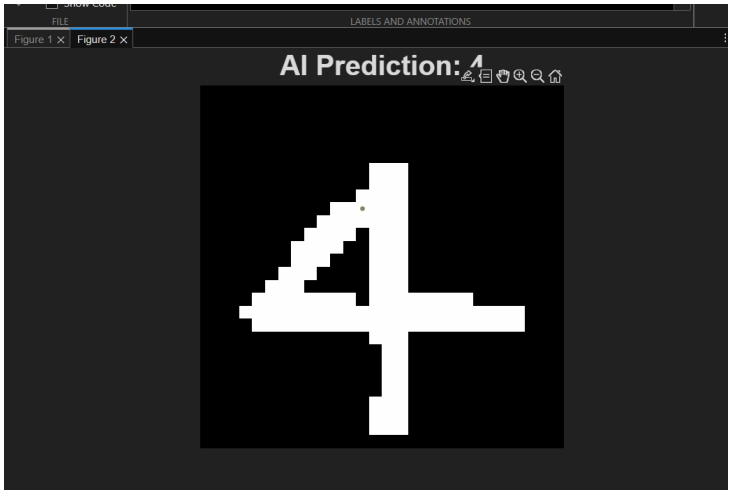


Figure 5.2: Real-time CNN prediction output for handwritten digit classification using MATLAB hardware-in-the-loop setup.

5.4 Power Consumption Analysis

Power consumption is a critical metric for edge devices. The total power consumption of the system was observed to be significantly optimized due to the implemented design techniques.

- The ARM processor consumes a major portion of total power
- FPGA-based CNN computation consumes comparatively less power
- Clock gating reduces dynamic switching activity

The use of Sparsity-Aware Dynamic Clock Gating (SADCG) effectively reduces unnecessary computations, leading to improved energy efficiency.

Table 5.1: Power Consumption Comparison

Power Metric	**Standard Baseline**	**With SADCG**
Static Power	0.142 W	0.142 W
Dynamic Power (PL)	0.875 W	0.510 W
PS Power (ARM)	1.350 W	1.350 W
Total Power	**2.367 W**	**2.002 W**

5.5 Memory and Resource Utilization

The FPGA implementation demonstrates efficient utilization of hardware resources.

- On-chip memory (BRAM) is effectively used for storing intermediate data
- Shift buffers reduce external memory access

- Parallel Processing Elements improve computational throughput

Efficient memory usage plays a crucial role in improving overall system performance and reducing latency.

5.6 Performance Evaluation

The performance of the proposed system is evaluated based on the following metrics:

- **Latency:** Reduced due to pipelined architecture
- **Throughput:** Increased using parallel processing
- **Power Efficiency:** Improved using clock gating and sparsity exploitation

The system achieves real-time classification with optimized hardware utilization, making it suitable for edge AI applications.

5.7 Comparison with Existing Systems

Compared to traditional CPU and GPU-based implementations, the proposed FPGA-based accelerator offers several advantages:

- Lower power consumption
- Reduced latency
- Improved energy efficiency
- Custom hardware optimization

While GPUs provide high computational power, they are not suitable for embedded applications due to high power requirements. The proposed FPGA solution provides a balanced trade-off between performance and efficiency.

5.8 Discussion

The results demonstrate that the proposed CNN hardware accelerator successfully achieves its design goals. The integration of parallel processing, memory optimization, and clock gating significantly enhances system performance.

The system is capable of performing real-time inference with reduced power consumption, making it suitable for deployment in embedded and edge environments.

However, the performance of the system depends on the efficiency of hardware design and the level of quantization applied. Further improvements can be achieved by optimizing the architecture and exploring advanced neural network models.

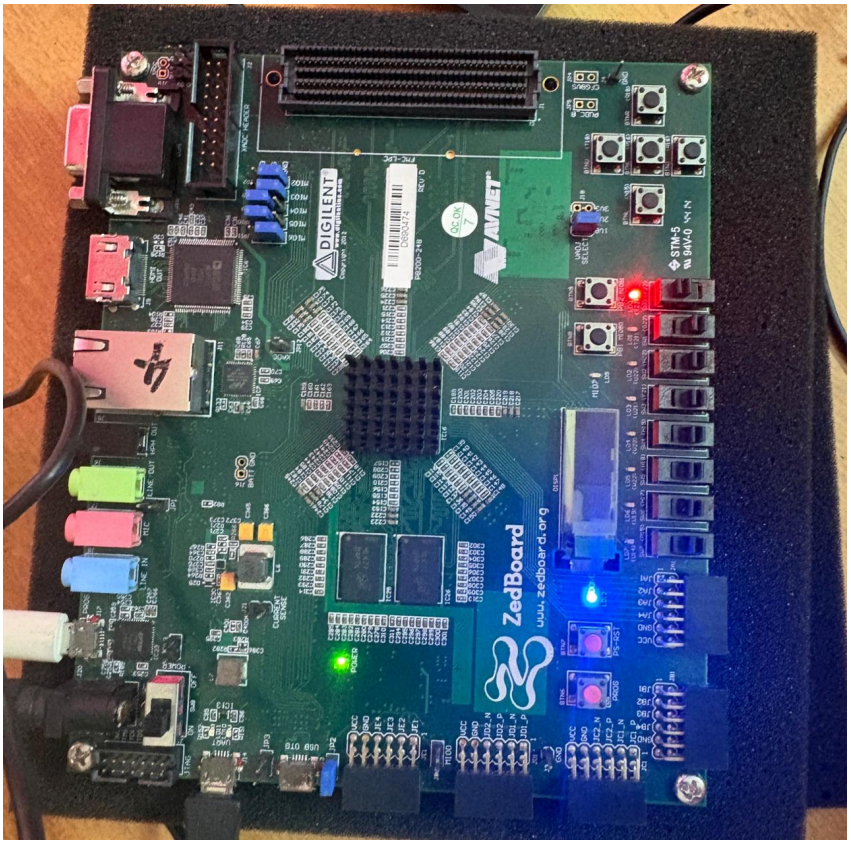


Figure 5.3: Final hardware implementation of the CNN accelerator on Zynq-7000 FPGA development board.

Chapter 6

CONCLUSION AND FUTURE SCOPE

6.1 Conclusion

In this project, an energy-efficient hardware accelerator for Convolutional Neural Network (CNN) inference has been successfully designed and implemented on the Xilinx Zynq-7000 System-on-Chip (SoC). The proposed system effectively addresses the major challenges associated with deploying deep learning models on edge devices, including high power consumption, computational complexity, and memory bandwidth limitations.

A hybrid hardware-software co-design approach was adopted, where preprocessing and control operations were handled by the ARM Cortex-A9 processor, while the computationally intensive CNN operations were executed in FPGA fabric. This partitioning significantly improved system efficiency and reduced processing latency.

The implementation incorporated several optimization techniques such as parallel Processing Elements (PEs), shift buffer-based memory reuse, pipelined architecture, and post-training quantization. Additionally, Sparsity-Aware Dynamic Clock Gating (SADCG) was utilized to reduce dynamic power consumption by skipping redundant computations.

The system was successfully validated using the MNIST dataset, demonstrating accurate real-time handwritten digit classification. The results confirm that the proposed architecture achieves a balance between performance, resource utilization, and energy efficiency.

Overall, this work highlights the potential of FPGA-based CNN accelerators as a practical and scalable solution for edge AI applications, offering significant advantages over traditional CPU and GPU implementations.

6.2 Future Scope

Although the proposed system achieves efficient CNN inference, several improvements and extensions can be explored in future work:

- **Support for Complex Datasets:** Extending the architecture to handle more complex datasets such as CIFAR-10 or ImageNet for advanced image classification tasks.

- **Advanced Neural Network Models:** Implementing deeper and more sophisticated architectures such as ResNet or MobileNet for improved accuracy.
- **Dynamic Reconfiguration:** Utilizing partial reconfiguration in FPGA to adapt the architecture dynamically based on application requirements.
- **Further Power Optimization:** Exploring voltage scaling, adaptive clocking, and approximate computing techniques for additional energy savings.
- **Edge Deployment:** Integrating the system into real-world edge devices such as smart cameras, healthcare monitoring systems, and IoT applications.
- **Hardware Acceleration of Training:** Extending the design to support on-chip training or online learning capabilities.

These future enhancements can further improve the scalability, flexibility, and performance of FPGA-based AI accelerators, making them more suitable for next-generation intelligent systems.

Bibliography

[1] Y. LeCun et al., "Gradient-based learning applied to document recognition," Proceedings of the IEEE, 1998.

[2] Xilinx Inc., "Zynq-7000 SoC Technical Reference Manual," 2020.

[3] Y. LeCun, "The MNIST database of handwritten digits," 1998.

[4] S. Gupta et al., "Deep Learning with Limited Numerical Precision," ICML, 2015.

[5] Various Authors, "FPGA-based CNN Accelerators for Edge Computing," IEEE Journals.